



版本: 1.0.1 2021 年 5 月

AnyCloud39EV330 平台

Hi3881 模组驱动移植指导手册

声 明

本手册的版权归广州安凯微电子股份有限公司所有，受相关法律法规的保护。未经广州安凯微电子股份有限公司的事先书面许可，任何人不得复制、传播本手册的内容。

本手册所涉及的知识产权归属广州安凯微电子股份有限公司所有（或经合作商授权许可使用），任何人不得侵犯。

本手册不对包括但不限于下列事项担保：适销性、特殊用途的适用性；实施该用途不会侵害第三方的知识产权等权利。

广州安凯微电子股份有限公司不对由使用本手册或执行本手册内容而带来的任何损害负责。

本手册是按当前的状态提供参考，随附产品或本书内容如有更改，恕不另行通知。

联 系 方 式

广州安凯微电子股份有限公司

地址：广州黄埔区知识城博文路 107 号安凯微电子 H 大厦

电话: (86)-20-3221 9000

传真: (86)-20-3221 9258

邮编: 510663

销售热线:

(86)-20-3221 9499

电子邮箱:

sales@anyka.com

主页:

<http://www.anyka.com>

版本变更说明

以下表格对于本文档的版本变更做一个简要的说明。版本变更仅限于技术内容的变更，不包括版式、格式、句法等的变更。

版本	说明	完成日期
V1.0.0	首次编辑	2021 年 4 月
V1.0.1	增加驱动传参、日志打印等级设置	2021 年 5 月

目录

1	概述	6
2	移植说明	7
2.1	ANYCLOUDV330 PDK 版本	7
2.2	ANYCLOUDV330 硬件	7
2.3	Hi3881 驱动源码	7
2.4	Hi3881 模组	7
2.5	硬件处理	8
3	驱动移植	10
3.1	Hi3881 驱动编译	10
3.1.1	配置工具链	10
3.1.2	配置 Hi3881 固件路径	11
3.1.3	增加探卡接口	12
3.1.4	驱动编译	13
3.2	DTS 文件配置	13
3.2.1	配置 mmc idle 属性	14
3.2.2	配置 wifi_en 属性	14
3.2.3	优化 mmc1 节点	14
3.2.4	DTB 编译	15
3.3	AK_MCI 驱动配置	15
3.3.1	ak_mci 源码修改	15
3.3.2	ak_mci 驱动编译	16
3.4	驱动加载	16
3.4.1	mmc 驱动加载	16
3.4.2	Hi3881 模组驱动加载	17
3.4.3	带参数的驱动加载	19
3.5	驱动日志打印等级设置	19
3.6	驱动卸载	20
4	WIFI 配网操作	21
4.1	STATION 模式配网	21
4.2	AP 模式配网	21
5	IPERF 简易测速	24

5.1	STATION 模式测速	24
5.1.1	开发板当 server	24
5.1.2	开发板当 client	24
5.2	AP 模式测速	25
5.2.1	开发板当 server	25
6	对接过程中遇到的部分问题	26
6.1	问题 1	26
6.2	问题 2	26
6.3	问题 3	27
6.4	案例分析	27
7	附录:	28

1 概述

鉴于使用 Hi3881 SDIO WIFI 模组（以下简称 Hi3881 模组）的客户越来越多，现提供一份基于 AnyCloud39EV330 平台的 Hi3881 模组驱动移植手册，方便客户参考。

2 移植说明

2.1 AnyCloudV330 PDK 版本

基于 AnyCloud39EV330_PDK_V1.01_PATCH1-2 版本进行移植，其他版本供参考。

2.2 AnyCloudV330 硬件

使用 AnyCloudV330 标准硬件进行移植，其他硬件供参考。

开发板：EVB_MBD_AK391XEV33X_V1.0.1

核心板：EVB_CBD_AK3916EV33X_V1.0.2

WIFI 小板：C500_WIFI_SDIO_14Pin_V1.0.0

2.3 Hi3881 驱动源码

目前已知的源码包如表 1.1 所示，此教程使用 2020-11-27 释放的驱动源码包，相关源码包找 FAE 进行申请，其他版本的驱动源码供参考。

表 1.1

序号	源码包名	释放日期	备注
1	Hi3881V100R001C00SPC021.zip	2020-11-27	B002
2	Hi3881V100R001C00SPC021B006_ak.tar.gz	2020-03	
3	Hi3881V100R001C00SPC021.tgz	2021-04-02	未能正常使用

2.4 Hi3881 模组

Hi3881 是海思的 SDIO WIFI 芯片，实际对接的是第三方根据 Hi3881 基础硬件生产的 SDIO WIFI 模组，不同厂家生产的模组有细微差异，但 SDIO 引脚都是标准的。本次移植采用的是涂鸦生产的 Hi3881 SDIO WIFI 模组（无特殊说明，以下都以 Hi3881 模组代指涂鸦的 WIFI 模组），其他厂家的 Hi3881 模组供参考，Hi3881 模组引脚图如图 1.1 所示，与常规的 RTL8189FTV 44Pin 模组 Pin2Pin。

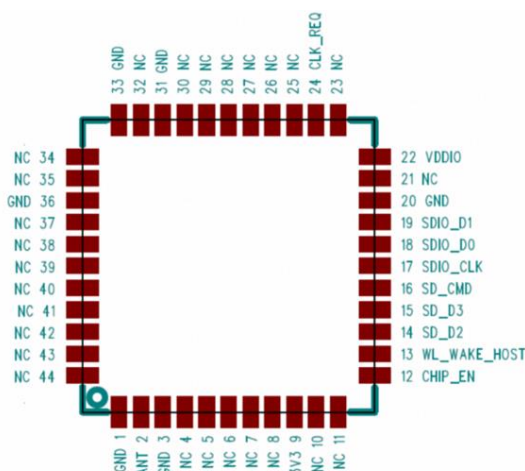


图 1.1 涂鸦 Hi3881 SDIO WIFI 模组引脚图

2.5 硬件处理

由于安凯开发板 WIFI 模组没有与 Hi3881 模组的 Pin2Pin 的小板，需要采用飞线的方式焊接到 C500_WIFI_SDIO_14Pin_V1.0.0 小板上，飞线处理找硬件进行支持。其他硬件供参考。

Wifi 小板原理图，如图 1.2 所示：

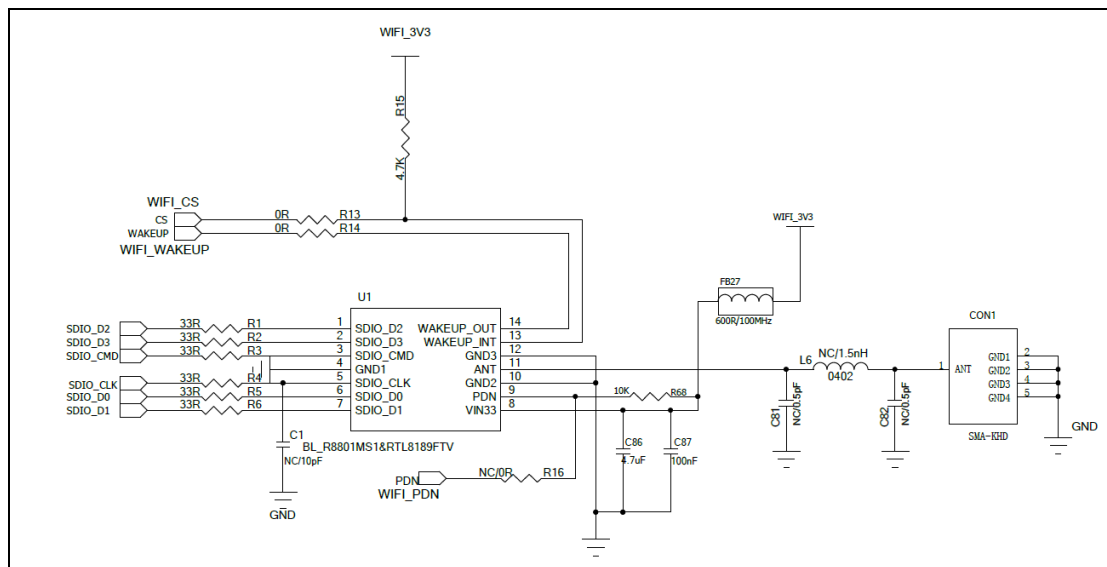


图 1.2 Wifi 小板原理图

详细原理图，可以查看 PDK 资料，路径：PDK_V1.01\PDK\HDK\开发板资料\参考原理图\C500_WIFI_SDIO_14Pin_V1.0.0.pdf

如表 1.2 所示，是飞线用的连线表，供硬件工程师焊接飞线时使用。

连线表 1.2:

序号	C500_WIFI_SDIO_14Pin_V1.0.0 小板 焊盘序号	Hi3881 模组 引脚序号	备注
1	1 - SDIO_D2	14 - SD_D2	
2	2 - SDIO_D3	15 - SD_3	
3	3 - SDIO_CMD	16 - SD_CMD	
4	4 - GND1	-	NC
5	5 - SDIO_CLK	17 - SDIO_CLK	
6	6 - SDIO_D0	18 - SDIO_D0	
7	7 - SDIO_D1	19 - SDIO_D1	
8	8 - VIN33	9 - 3V3 22 - VDDIO	
9	9 - PDN	12 - CHIP_EN	
10	10 - GND2	20 - GND	
11	11 - ANT	2 - ANT	
12	12 - GND3	3 - GND	
13	13 - WAKEUP_INT	-	NC
14	14 - WAKEUP_OUT	13 - WL_WAKE_HOST	

3 驱动移植

进行 Hi3881 模组移植前，默认大家已经使用 PDK 自动编译脚本编译过一次固件了，以下移植步骤都是基于编译过的固件上进行的。

如果没有编译过，请参考《AnyCloud39EV330 PDK 用户开发手册_V1.0.1.pdf》文档第 3.2 小节进行编译。

3.1 Hi3881 驱动编译

海思 Hi3881 驱动源码（Hi3881V100R001C00SPC021.zip）是针对安凯 V300 和 V330 平台进行了专门适配的源码，需要进行相应的配置后才能编译出可以用的 ko。

3.1.1 配置工具链

驱动源码包解压后，打开根目录的 Makefile 文件，定义内核 build 路径，同时定义 AnyCloud39EV330 工具链。注：红色部分为修改点（差异点）。

```
1 KDIR=/home/licheng/anyka/svn/v330/PDK_V1.01-2_new/PDK_V1.01-2/SDK/os/bd
2 AK_CROSS_COMPILE=arm-anykav500-linux-uclibcgnueabi-
```

同时配置驱动安装目录。

```
1 PREFIX = $(PRJ_ROOT)/_install
```

指定编译工具链为 CROSS_COMPILE=\${AK_CROSS_COMPILE}。

```
1 make -C ${WIFI_DRIVER_DIR} WIFI_DRIVER_DIR=${WIFI_DRIVER_DIR}
  CROSS_COMPILE=${AK_CROSS_COMPILE} HISILICON_PLATFORM=$(HISILICON_PLATFORM) KDIR=${KDIR} clean; \
2 make -C ${WIFI_DRIVER_DIR} WIFI_DRIVER_DIR=${WIFI_DRIVER_DIR}
  CROSS_COMPILE=${AK_CROSS_COMPILE} HISILICON_PLATFORM=$(HISILICON_PLATFORM) KDIR=${KDIR} -j16
```

为所有 make 命令指定编译工具链为 CROSS_COMPILE=\${AK_CROSS_COMPILE}。

```
1 $(MAKE) -C ${WIFI_DRIVER_DIR} WIFI_DRIVER_DIR=${WIFI_DRIVER_DIR}
  CROSS_COMPILE=${AK_CROSS_COMPILE} HISILICON_PLATFORM=$(HISILICON_PLATFORM) KDIR=${KDIR} clean
```

文件 driver/Makefile，将编译路径及交叉编译器，修改为如下内容，这里使用“?”赋值，顶层 Makefile 没有配置此变量才生效，实际可以不配置。

```
1 KDIR ?= /home/licheng/anyka/svn/v330/PDK_V1.01-2_new/PDK_V1.01-2/SDK/os/bd
2 AK_CROSS_COMPILE ?= arm-anykav500-linux-uclibcgnueabi-
  CROSS_COMPILE ?= ${AK_CROSS_COMPILE}
```

3.1.2 配置 Hi3881 固件路径

与常规 WIFI 模组不同，Hi3881 模组是加载驱动的时候需要向 Hi3881 芯片下载固件。常规 WIFI 模组出厂是自带固件的，WIFI 模组自带固件，可以省下产品 Flash 内存。加载驱动时再下载固件，可以降低 WIFI 模组成本。默认的固件路径与我们放置的路径不一致，需要修改。

固件路径相关的主要文件是 wifi_cfg，源码包里面默认提供了三份 wifi_cfg 文件。三份 wifi_cfg 文件对应的路径是：

- ✧ firmware/wifi_cfg/ce/wifi_cfg
- ✧ firmware/wifi_cfg/customer/wifi_cfg
- ✧ firmware/wifi_cfg/fcc/wifi_cfg

在《Hi3881V100 Linux 软件 使用指南.pdf》中有关于 bin 固件及 wifi_cfg 的描述：

- 编译的 hi3881_fw.bin 有业务 bin 和产测 bin 之分，产测 bin 需要重命名为 hi3881_mfg_fw.bin，置于单板/vendor/firmware/hisilicon 目录下（与 hi3881_fw.bin 存放目录相同）。
- 如果模组厂家更改了 WiFi cfg 参数，则需要确保终端用户最终编译的镜像中的 wifi_cfg 替换为修改后的 wifi_cfg 文件。
- 不同的定制化文件 firmware/wifi_cfg 适用对象不同：
- ce 文件夹下的 firmware/wifi_cfg：适用于模组厂家，功率符合 CE 和无委认证法规要求。
- customer1 文件夹下的 firmware/wifi_cfg：适用于 chip on board 厂家，功率符合 CE 和无委认证法规要求。
- fcc 文件夹下的 firmware/wifi_cfg：适用于模组厂家和 chip on board 厂家，功率符合 FCC 认证法规要求。

将这三份 wifi_cfg 文件有包含 Hi3881 模组固件的路径，此路径需要与存放在根文件系统的固件路径一致，本手册将固件路径放置到/usr/local 目录，即将第 9 行都修改为 /usr/local 目录。

```
1 ADDR_FILE_FILES=1,0xe8800,/usr/local/firmware/hisilicon/hi3881_fw.bin;
```

使用时只需要其中一份 wifi_cfg 文件即可，这里将 customer1 文件夹下的 wifi_cfg 文件放置到/usr/local 目录。

在 driver/oal/plat_firmware.c 文件，将 wifi_cfg 配置文件路径修改为/usr/local 目录。

```
1 #define WIFI_CFG_C01_PATH
   "/usr/local/firmware/hisilicon/wifi_cfg"
```

在同一文件将 hi1131h_demo_non_rom.bin 文件的路径修改为/usr/local 目录。

```
1  #define WIFI_FIRMWARE_FILE_BIN
   "/usr/local/firmware/hisilicon/hi1131h_demo_non_rom.bin"
```

同一文件将 hi3881_mfg_fw.bin 文件的路径修改为/usr/local 目录。

```
1  path = "/usr/local/firmware/hisilicon/hi3881_mfg_fw.bin";
```

hi1131h_demo_non_rom.bin 和 hi3881_mfg_fw.bin 是产测用固件，源码包中实际没有，后面产测可能会用到，这里为了统一，将此 bin 路径指定到/usr/local 目录。

3.1.3 增加探卡接口

Hi3881 模组属于 SDIO 设备，在加载驱动的过程中会去探测连接主控的第几个 SDIO (MMC) 接口，并探测该接口上连接的 Hi3881 模组是否在线，探测成功后且模组在线才能正常安装驱动。注：此接口只有 Hi3881 需要，其他 SDIO wifi 不需要此接口。

在 driver/oal/oal_sdio_host.c 文件，sdio_card_detect_change 接口中，加入安凯的探卡接口。

```
1  hi_void sdio_card_detect_change(hi_s32 val)
2  {
3  #if (_PRE_OS_VERSION_LINUX == _PRE_OS_VERSION)
4      ak_sdio_rescan(val);
5  #else
6      hi_s32 ret = hisi_sdio_rescan(val);
7      if (ret != HI_ERR_SUCCESS) {
8          oam_error_log0(0, 0, "sdio rescan failed.");
9      }
10 #endif
11 }
```

Hi3881 模组驱动，属于外部编译的，接口只需声明后即可正常编译驱动。在 driver/oal/oal_sdio.h 文件，使用 extern 关键字声明此探卡接口。

```
1  #if (_PRE_OS_VERSION_LINUX == _PRE_OS_VERSION)
2  extern void ak_sdio_rescan(int slot);
3  #else
4  extern int hisi_sdio_rescan(int slot);
5  #endif
```

注：1.此小节修改点已经包含在海思适配好的 Hi3881 源码包当中，此处只作记录，方便其他版本的驱动源码移植。

注：2.具体的探卡接口在 ak_mci 驱动里面实现，见 3.3 小节。

3.1.4 驱动编译

经过以上几点修改后即可编译 Hi3881 模组驱动了，在源码根目录中执行：

```
1 make linux_driver
```

编译日志会提示 `ak_sdio_rescan` 接口没有定义，属于正常提示，因为我们平台的 `ak_mci` 驱动也是外部编译的，没有在内核里面。

```
1 WARNING: "ak_sdio_rescan"
  [/home/licheng/anyka/cp/v330/hi3881/Hi3881V100R001C00SPC021B002/drive
  r/hi3881.ko] undefined!
```

编译没有报错，则会在 `driver` 目录下生成 `hi3881.ko` 文件。将生成 `hi3881.ko` 拷贝到根文件系统 `platform/rootfs/ko/external` 目录。

```
1 cp -r driver/hi3881.ko platform/rootfs/ko/external/
```

同时将对应的固件 `hi3881_fw.bin` 及 `wifi_cfg` 配置文件拷贝到根文件系统指定目录。并将拷贝到根文件系统的 `rootfs/utils/usr/local/firmware/wifi_cfg` 文件夹移至为备份目录 `wifi_cfg_bak`，否则会与 `wifi_cfg` 文件同名。将 `rootfs/utils/usr/local/firmware/wifi_cfg_bak/customer/wifi_cfg` 文件取出，放置 `rootfs/utils/usr/local/firmware` 目录。示意如下，供参考。

```
1 mkdir -p platform/rootfs/utils/usr/local/firmware/hisilicon
2 cp -r firmware/* platform/rootfs/utils/usr/local/firmware/hisilicon
3 mv platform/rootfs/utils/usr/local/firmware/wifi_cfg
  platform/rootfs/utils/usr/local/firmware/wifi_cfg_bak
4 cp -r
  platform/rootfs/utils/usr/local/firmware/wifi_cfg_bak/customer/wifi_c
  fg platform/rootfs/utils/usr/local/firmware
```

注：命令供参考，文档为了排版，省略了部分目录信息，请根据你自己的实际路径进行执行。

3.2 DTS 文件配置

使用 PDK 默认的 DTS 文件(`EVB_CBD_AK3916EV331_V1.0.0.dts`)，复位按键复位主控，再次加载 Hi3881 驱动会出现加载异常的现象。主要原因是 Hi3881 模组在执行 `insmod hi3881.ko` 前，没有对模组进行硬复位或者软复位操作。而我们主控复位过程中，模组没有掉电，一直处于工作状态中，虽然 `ak_mic` 驱动有失能再使能 `Wifi_Power` 操作，但我们的 MMC 引脚会持续给 Hi3881 供电，导致再次加载驱动会失败。

需要在 DTS 文件中添加 `mmc idle` 属性，使 MMC 引脚在空闲时为普通 GPIO 低电平状态，这样能保证在失能 `Wifi_Power` 时，MMC 引脚不会持续给 Hi3881 模组供电。

3.2.1 配置 mmc idle 属性

在 EVB_CBD_AK3916EV331_V1.0.0.dts 文件，添加空闲时 mmc io 属性。

```
1      mci1_idle_pins:mci1_idle_pins
2      {
3          anyka,pins = <41 42 43 44 45 46>;
4          anyka,function = <0>;
5          anyka,pull = <0x00000000>;
6      };
```

3.2.2 配置 wifi_en 属性

在 EVB_CBD_AK3916EV331_V1.0.0.dts 文件，开启或添加 wifi_en 属性。

```
1      wifi_en_pins:wifi_en_pins{
2          anyka,pins = <77>;
3          anyka,function = <0>;
4          anyka,pull = <0x00000010>;
5      };
```

3.2.3 优化 mmc1 节点

在 EVB_CBD_AK3916EV331_V1.0.0.dts 文件，对 mmc1 节点进行优化。

```
1      &mmc1 {
2          bus-width = <0x4>;
3          max-frequency = <50000000>;
4          /*
5           * detect mode:
6           * (1) non-removable;
7           * (2) detect-gpio/detect-gpios: card detect using gpio
8           * for example:
9           * detect-gpio = <&gpio 76 1>; or detect-gpios = <&gpio
10          76 1>;
11          * (3) cd_clk: card detect using MCLK
12          */
13          non-removable;
14          /* if support highspeed, using cap-sd-highspeed */
15          cap-sd-highspeed;
16          /* if support sdio, using cap-sdio-irq*/
17          cap-sdio-irq;
18          pinctrl-names = "default","idle";
19          /* pinctrl-0 = <&mci1_pins>; */
20          /*
21           * support sdio wifi circuit enable function
22           */
23          pinctrl-0 = <&mci1_pins>,<&wifi_en_pins>;
24          pinctrl-1 = <&mci1_idle_pins>;
25          power-pins = <&gpio 77 1>; //power-pins is which wifi_en_pins
26          describes
27
28          status = "okay";
29      };
```

3.2.4 DTB 编译

通过以上步骤修改完 dts 后，需要重新编译 dts 生成可用的 dtb 文件。在内核源码目录执行如下 make 命令，既可以编译出新的 dtb 文件。

```
1 make O=../bd dtbs -j16
```

将编译生成的 dtb 文件拷贝到 burnttool 文件夹。

```
1 cp -
r ../bd/arch/arm/boot/dts/EVB_CBD_AK3916EV331_V1.0.0.dtb ../../tools/
burnttool
```

注：命令供参考，文档为了排版，省略了部分目录信息，请根据你自己的实际路径进行执行。

3.3 ak_mci 驱动配置

因为 Hi3881 驱动有探卡行为，参考海思的 mmc 源码实现 ak_sdio_rescan 接口。本手册 3.2 节 DTS 配置中有说明，我们需要在 DTS 文件中添加 mmc idle 属性，要使该属性生效，当前 PDK 版本的 ak_mci 驱动需要更新到 1.0.09 版本或以上，对应版本的源码或者驱动可找 FAE 进行申请。

3.3.1 ak_mci 源码修改

在 1.0.09 版本的 ak_mci.c 源码，添加全局变量及结构体声明：

```
1 unsigned int slot_index = 0;
2 struct mmc_host *mci_host[MCI_SLOT_NUM] = {NULL};
```

同一文件，在代码空白处添加 ak_sdio_rescan() 接口。并采用模块导出符号 EXPORT_SYMBOL_GPL，将接口导出给外部(hi3381.ko)调用。

```
1 int ak_sdio_rescan(int slot)
2 {
3     struct mmc_host *mmc = mci_host[slot];
4     if (!mmc) {
5         pr_err("invalid mmc, please check the argument\n");
6         return -EINVAL;
7     }
8     mmc_detect_change(mmc, 0);
9     return 0;
10 }
11 EXPORT_SYMBOL_GPL(ak_sdio_rescan);
```

同一文件，在接口 akmci_probe 中，添加如下语句，将 host->mmc 参数保存到 mci_host 结构体数组中。

```
1 // for test
2 //DDREGS(host);
3 mci_host[slot_index++] = host->mmc;
4 return 0;
```

在 ak_mci.h 文件，定义 MMC 接口个数。

```
1 #define MAX_MCI_REQ_SIZE    (65536)
2
3 #define MCI_SLOT_NUM        2
4
5 /* 12 fifo limit to 512 bytes */
```

3.3.2 ak_mci 驱动编译

修改完毕后重新编译 ak_mci 驱动，使用外部编译命令，并把编译好的 ak_mci.ko 拷贝到根文件系统 ko 目录。

```
1 make KSRC=../SDK/os/bd
2 cp -r ak_mci.ko ../SDK/platform/rootfs/ko/external/
```

注：命令供参考，文档为了排版，省略了部分目录信息，请根据你自己的实际路径进行执行。

3.4 驱动加载

在调试驱动阶段，尽量使用有线 nfs 挂载的方式进行调试，提高调试效率，减少不必要文件拷贝。如果没有网卡，可以采用 usb 转有线网卡的方式进行，使用方式参考《Anyka V330(V500) USB 转有线网卡使用文档.pdf》文档，此文档可找 FAE 进行申请。

3.4.1 mmc 驱动加载

要正常使用 Hi3881 模组，需要先加载 mmc 驱动，mmc 除了与 SDIO 设备关联还与 T 卡相关，如果有卸载 mmc 驱动的情况，注意规避对 T 卡的影响。

```
1 modprobe mmc_core.ko
2 modprobe mmc_block.ko
3 insmod /usr/modules/ak_mci.ko
```

硬件连接没有问题，放开内核打印或者执行 dmesg 会有“mmc1: new high speed SDIO card at address 0001”打印，存在此打印信息，说明硬件上 SDIO 连接没有问题，可以被 mmc 驱动正常识别到。如果没有此打印信息，要认真排查一下硬件。

正常识别 SDIO 设备的日志打印如下：


```

1  akmci 20108000.mmc1: akmci_probe : SDIO
2  of_get_named_gpiod_flags: parsed 'power-pins' property of node
   '/soc/mmc1@20108000[0]' - status (0)
3  [default setting] enable sdio_card circuit using gpio77 with
   gpio_val 1
4  of_get_named_gpiod_flags: can't parse 'rst-pins' property of node
   '/soc/mmc1@20108000[0]'
5  of_get_named_gpiod_flags: can't parse 'wp-gpios' property of node
   '/soc/mmc1@20108000[0]'
6  of_get_named_gpiod_flags: can't parse 'wp-gpio' property of node
   '/soc/mmc1@20108000[0]'
7  akmci 20108000.mmc1: lookup for GPIO wp failed
8  akmci 20108000.mmc1: gclk = 160000000 Hz. div_l=199, div_h=199
9  akmci 20108000.mmc1: ios->clock = 400000 Hz. host->bus_clock =
   400000 Hz
10 akmci 20108000.mmc1: Mci1 Interface.using l2dma.sdio irq detect
   mode:plugin alway.
11 akmci 20108000.mmc1: gclk = 160000000 Hz. div_l=1, div_h=1
12 akmci 20108000.mmc1: ios->clock = 50000000 Hz. host->bus_clock =
   40000000 Hz
13 akmci 20108000.mmc1: gclk = 160000000 Hz. div_l=1, div_h=1
14 akmci 20108000.mmc1: ios->clock = 50000000 Hz. host->bus_clock =
   40000000 Hz
15 mmc1: queuing unknown CIS tuple 0x91 (5 bytes)
16 mmc1: new high speed SDIO card at address 0001

```

3.4.2 Hi3881 模组驱动加载

在主控正确识别到 Hi3881 模组(SDIO 设备)后, 就可以进行 hi3881.ko 的加载了, 配网依赖于 cfg80211, 在加载 hi3881.ko 时要先加载 cfg80211.ko。

```

1  modprobe cfg80211.ko
2  insmod /usr/modules/hi3881.ko

```

放开内核打印或者执行 dmesg 会有如下日志打印。

```

1  [VERSION]:Hi3881V100R001C00SPC021 2020-11-11 15:10:00
2  oam_main_init SUCCESSFULLY!
3  wal_customize_init SUCCESSFULLY!
4  oal_main_init SUCCESSFULLY
5  frw_main_init SUCCESSFULLY!
6  hi_wifi_plat_init SUCCESSFULLY
7  {alloc 131072 bytes tx_flat_buff!}
8  {alloc 131072 bytes rx_flat_buff!}
9  hcc_hmac_init SUCCESSFULLY
10 plat_firmware_init SUCCESSFULLY
11 Device is Ready!
12 wlan_pm_open SUCCESSFULLY!!
13 hmac_main_init SUCCESSULLY
14 firmware_sync_cfg_paras_to_wal_customize SUCCESSFULLY
15 wal_main_init SUCCESSFULLY
16 hi_wifi_host_init SUCCESSFULLY
17 WiFi driver init SUCCESSFULLY!

```

若与日志信息一致，则表明驱动加载正常。使用“ifconfig -a”查看 wlan0 节点是否正常生成。关键节点与红色一致，说明节点已经正常生成，驱动移植完成。

```

1  [root@anyka ~]$ ifconfig -a
2  Hisilicon0 Link encap:Ethernet  HWaddr 00:00:00:00:00:00
3          BROADCAST MULTICAST  MTU:1500  Metric:1
4          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
5          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
6          collisions:0 txqueuelen:1000
7          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
8
9
10 ap0      Link encap:Ethernet  HWaddr FA:11:31:BB:53:5D
11          inet addr:172.14.10.1  Bcast:172.14.255.255
12          Mask:255.255.0.0
13          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
14          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
15          TX packets:10 errors:0 dropped:0 overruns:0 carrier:0
16          collisions:0 txqueuelen:1000
17          RX bytes:10 (10.0 B)  TX bytes:10 (10.0 B)
18
19 eth0      Link encap:Ethernet  HWaddr 72:58:01:65:48:A5
20          inet addr:172.16.2.124  Bcast:172.16.255.255
21          Mask:255.255.0.0
22          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
23          RX packets:266 errors:0 dropped:52 overruns:0 frame:0
24          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
25          collisions:0 txqueuelen:1000
26          RX bytes:27608 (26.9 KiB)  TX bytes:684 (684.0 B)
27          Interrupt:16 Base address:0xc000
28
29 lo        Link encap: Local Loopback
30          inet addr:127.0.0.1  Mask:255.0.0.0
31          UP LOOPBACK RUNNING  MTU:65536  Metric:1
32          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
33          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
34          collisions:0 txqueuelen:1
35          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
36
37
38 wlan0     Link encap:Ethernet  HWaddr FA:11:31:BB:53:5C
39          BROADCAST MULTICAST  MTU:1500  Metric:1
40          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
41          TX packets:10 errors:0 dropped:0 overruns:0 carrier:0
42          collisions:0 txqueuelen:1000
43          RX bytes:10 (10.0 B)  TX bytes:10 (10.0 B)

```

3.4.3 带参数的驱动加载

Hi3881 驱动源码是支持加载 ko 时传参的，使用 `g_mode=x` 来选择驱动支持什么模式，不带参数或者 `g_mode=2` 默认支持 STA 模式和 AP 模式共存，`g_mode=1` 支持 AP 模式，`g_mode=0` 支持 STA 模式。

STA 模式和 AP 模式共存

```
1 insmod /usr/modules/hi3881.ko g_mode=2
```

AP 模式

```
1 insmod /usr/modules/hi3881.ko g_mode=1
```

STA 模式

```
1 insmod /usr/modules/hi3881.ko g_mode=0
```

3.5 驱动日志打印等级设置

Hi3881 支持在加载驱动后设置打印等级，在 `/sys/hi3881_debug/loglevel` 中，通过设置 `hi3881_debug` 的值，可以设置 hi3881 驱动的日志打印等级，`cat /sys/hi3881/hi3881_debug` 有打印等级设置的相关描述。

```
1 cat /sys/hi3881_debug/loglevel
2 loglevel:
3 0      close log
4 1      ERROR
5 2      WARN
6 3      INFO
```

设置打印等级：

```
1 echo 3 > /sys/hi3881_debug/loglevel
```

客户的大部分问题都是出在驱动加载时候的，而日志打印等级设置是在驱动加载后的，无法观察到驱动加载时的打印。为了方便调试 Hi3881 驱动，希望能在加载驱动时传参设置打印等级。

驱动源码 `driver/wal/hi_wifi_driver_api.c` 文件中，在 `hi_u8 g_wifi_init_flag = HI_FALSE;` 语句之前，增加如下代码：

```
1 extern hi_u32 g_level_log;
2
3
4 /* insmod 设置 log_level */
5 int log_level = 1;
6 module_param(log_level, int, 0644);
7 int __get_log_level(hi_void)
```

```
8  {
9      return log_level;
10 }
```

在接口 hi_s32 hi_linux_wifi_init(hi_void)中，hi_u8 const user_res_num = 4;语句之后，增加如下代码：

```
1  g_level_log = __get_log_level();
```

重新编译驱动

```
1  make linux_driver
```

将编译好的驱动拷贝到根文件系统，使用如下命令加载并设置打印等级。

```
1  insmod /tmp/nfs/pdk101_2/hi3881.ko g_mode=0 log_level=3
```

设置之后，加载驱动就可以显示更多的信息，方便我们调试 hi3881 驱动。

注：patch 文件提供的驱动不支持日志传参，需要此功能的可以按照上述修改代码后重新编译。

3.6 驱动卸载

卸载 Hi3881 模组驱动执行如下命令即可

```
1  rmmod hi3881
2  rmmod cfg80211
```

如果有卸载 mmc 驱动的情况，T 卡使用会受到影响，T 卡识别也依赖于 mmc 驱动，应注意规避(或允许)T 卡不能正常工作的情况。

```
1  rmmod /usr/modules/ak_mci.ko
2  modprobe -r mmc_core.ko
3  modprobe -r mmc_block.ko
```

4 WIFI 配网操作

确认驱动正常加载，节点正常生成后，就可以进行配网了。

4.1 Station 模式配网

模组为 station 模式，直接使用 PDK 的 wpa_supplicant 工具就可以正常配网了，联网配置文件默认在 /etc/config 目录，修改目录下 wpa_supplicant.conf 文件的 wifi 名及 wifi 密码。

```
1 ctrl_interface=/var/run/wpa_supplicant
2 ap_scan=1
3 network={
4     ssid="WDR3800"
5     key_mgmt=WPA-PSK
6     pairwise=CCMP TKIP
7     group=CCMP TKIP WEP104 WEP40
8     psk="31415926535897"
9     scan_ssid=1
10 }
```

执行如下配网命令进行配网，并使用 udhcpc 命令获取 ip。

```
1 wpa_supplicant -Dnl80211 -i wlan0 -c
  /etc/config/wpa_supplicant.conf -B &
2 udhcpc -i wlan0 &
```

配网成功会有如下日志信息：

```
1 Successfully initialized wpa_supplicant
2 rfkill: Cannot open RFKILL control device
3 OK
4 random: wpa_supplicant: uninitialized urandom read (32 bytes read,
5 126 bits of entropy available)
6 random: nonblocking pool is initialized
7 udhcpc: started, v1.30.1
8 udhcpc: sending discover
9 udhcpc: sending discover
10 udhcpc: sending select for 192.168.10.189
11 udhcpc: lease of 192.168.10.189 obtained, lease time 43200
12 deleting routes
13 route: SIOCDELRT: No such process
14 adding dns 192.168.10.1
```

4.2 Ap 模式配网

由于 Hi3881 模组与常规 wifi 模组存在差异，需要修改 hostapd.conf 和 udhcp.conf 文件，打开 /etc 目录下的 hostapd.conf 文件，将 interface=wlan0 要改为 interface=ap0。

```

1 interface=ap0
2 ctrl_interface=/var/hostapd
3 ignore_broadcast_ssid=0
4 wmm_enabled=1
5 ht_capab=[SHORT-GI-20]
6 macaddr_acl=0
7 auth_algs=1
8 wpa_pairwise=TKIP CCMP
9 rsn_pairwise=CCMP
10 ssid=AK_IPC
11 wpa_passphrase=11111111
12 channel=1
13 hw_mode=g
14 ieee80211n=0
15 wpa=2
16 wpa_key_mgmt=WPA-PSK

```

同样的打开/etc 目录下的 udhcp.conf 文件，将 wlan0 修改为 ap0。

```

1 option domain local
2 option lease 864000
3 max_leases 52
4 start 172.14.10.1
5 end 172.14.10.52
6 interface ap0
7 opt dns 0.0.0.0
8 option subnet 255.255.255.0
9 opt router 172.14.10.1

```

两个文件修改完后，执行如下命令进行 AP 热点配置。

```

1 ifconfig ap0 172.14.10.1
2 mkdir -p /var/lib/misc/
3 touch /var/lib/misc/udhcpd.leases
4 udhcpd /etc/config/udhcpd.conf -S
5 hostapd /etc/config/hostapd.conf -B

```

AP 热点配置成功会有如下日志信息：

```

1 OK
2 Configuration file: /etc/config/hostapd.conf
3 rfkill: Cannot open RFKILL control device
4 Failed to create interface mon.ap0: -95 (Operation not supported)
5 ap0: Could not connect to kernel driver
6 Using interface ap0 with hwaddr fa:11:31:bb:53:5d and ssid "AK_IPC"
7 random: Cannot read from /dev/random: hostapd: uninitialized
urandom read (32 bytes read, 113 bits of entropy available)
8 ndom: Resource temporarily unavarandom: hostapd: uninitialized
urandom read (32 bytes read, 113 bits of entropy available)
ilable
9 random: Only 0/20 bytes random: hostapd: uninitialized urandom read
(4 bytes read, 113 bits of entropy available)
10 of strong random data available random: hostapd: uninitialized
urandom read (16 bytes read, 114 bits of entropy available)
11 from /dev/random

```

```
12 random: Not enough entropy pool available for secure operations
13 WPA: Not enough entropy in random pool for secure operations -
14 update keys later when the first station connects
15 ap0: interface state UNINITIALIZED->ENABLED
16 ap0: AP-ENABLED
```

5 Iperf 简易测速

5.1 Station 模式测速

5.1.1 开发板当 server

```

1  -----
2  Server listening on 5201
3  -----
4  Accepted connection from 192.168.10.1, port 51896
5  [ 5] local 192.168.10.196 port 5201 connected to 192.168.10.1 port
6  51898
7  [ ID] Interval            Transfer      Bandwidth
8  [ 5]  0.00-1.02      sec  1.99 MBytes  16.4 Mbits/sec
9  [ 5]  1.02-2.00      sec  1.96 MBytes  16.6 Mbits/sec
10 [ 5]  2.00-3.00      sec  1.63 MBytes  13.6 Mbits/sec
11 [ 5]  3.00-4.00      sec  1.62 MBytes  13.7 Mbits/sec
12 [ 5]  4.00-5.01      sec  1.99 MBytes  16.6 Mbits/sec
13 [ 5]  5.01-6.00      sec  1.92 MBytes  16.2 Mbits/sec
14 [ 5]  6.00-7.00      sec  1.93 MBytes  16.2 Mbits/sec
15 [ 5]  7.00-8.00      sec  1.79 MBytes  15.0 Mbits/sec
16 [ 5]  8.00-9.00      sec  1.68 MBytes  14.1 Mbits/sec
17 [ 5]  9.00-10.01     sec  2.21 MBytes  18.4 Mbits/sec
18 [ 5] 10.01-10.02     sec  43.8 KBytes  32.3 Mbits/sec
19 -----
20 [ ID] Interval            Transfer      Bandwidth
21 [ 5]  0.00-10.02     sec  0.00 Bytes   0.00 bits/sec sender
22 [ 5]  0.00-10.02     sec  18.8 MBytes 15.7 Mbits/sec receiver

```

5.1.2 开发板当 client

```

1  [root@anyka /tmp/nfs/pdk101_2/tools]$ iperf3 -c 192.168.10.1 -i 1 -
2  t 10
3  Connecting to host 192.168.10.1, port 5201
4  [ 4] local 192.168.10.196 port 35256 connected to 192.168.10.1
5  port 5201
6  [ ID] Interval            Transfer      Bandwidth  Retr  Cwnd
7  [ 4]  0.00-1.01      sec  2.06 MBytes  17.0 Mbits/sec  0   122 KBytes
8  [ 4]  1.01-2.01      sec  1.73 MBytes  14.5 Mbits/sec  14  59.4 KBytes
9  [ 4]  2.01-3.01      sec  2.46 MBytes  20.6 Mbits/sec  0  86.3 KBytes
10 [ 4]  3.01-4.01      sec  2.14 MBytes  17.9 Mbits/sec  5  80.6 KBytes
11 [ 4]  4.01-5.00      sec  2.64 MBytes  22.3 Mbits/sec  8  72.1 KBytes
12 [ 4]  5.00-6.02      sec  2.30 MBytes  19.0 Mbits/sec  0  94.7 KBytes
13 [ 4]  6.02-7.01      sec  1.91 MBytes  16.2 Mbits/sec  1  79.2 KBytes
14 [ 4]  7.01-8.01      sec  1.98 MBytes  16.7 Mbits/sec  3  74.9 KBytes
15 [ 4]  8.01-9.01      sec  2.78 MBytes  23.2 Mbits/sec  0  99.0 KBytes
16 [ 4]  9.01-10.02     sec  2.53 MBytes  21.1 Mbits/sec  1  87.7 KBytes
17 -----
18 [ ID] Interval            Transfer      Bandwidth  Retr
19 [ 4]  0.00-10.02     sec  22.5 MBytes 18.9 Mbits/sec  32 sender
20 [ 4]  0.00-10.02     sec  22.4 MBytes 18.8 Mbits/sec receiver
21 iperf Done.

```


5.2 Ap 模式测速

5.2.1 开发板当 server

```

1  [root@anyka /tmp/nfs/pdk101_2/tools]$ iperf3 -s
2  warning: this system does not seem to support IPv6 - trying IPv4
3  -----
4  Server listening on 5201
5  -----
6  Accepted connection from 192.168.0.20, port 51281
7  [ 5] local 192.168.0.1 port 5201 connected to 192.168.0.20 port
8  51282
9  [ ID] Interval            Transfer        Bandwidth
10 [ 5] 0.00-1.00 sec      1.70 MBytes    14.2 Mbits/sec
11 [ 5] 1.00-2.00 sec      1.73 MBytes    14.5 Mbits/sec
12 [ 5] 2.00-3.01 sec      1.43 MBytes    11.9 Mbits/sec
13 [ 5] 3.01-4.00 sec       976 KBytes     8.04 Mbits/sec
14 [ 5] 4.00-5.00 sec      1.31 MBytes    11.0 Mbits/sec
15 [ 5] 5.00-6.00 sec      1.18 MBytes     9.86 Mbits/sec
16 [ 5] 6.00-7.00 sec      1.83 MBytes    15.3 Mbits/sec
17 [ 5] 7.00-8.00 sec      1.69 MBytes    14.1 Mbits/sec
18 [ 5] 8.00-9.00 sec      2.03 MBytes    17.0 Mbits/sec
19 [ 5] 9.00-10.00 sec     2.10 MBytes    17.6 Mbits/sec
20 [ 5] 10.00-10.05 sec     125 KBytes     19.6 Mbits/sec
21 - - - - -
22 [ ID] Interval            Transfer        Bandwidth
23 [ 5] 0.00-10.05 sec       0.00 Bytes      0.00 bits/sec   sender
24 [ 5] 0.00-10.05 sec     16.1 MBytes    13.4 Mbits/sec  receiver

```

6 对接过程中遇到的部分问题

6.1 问题 1

加载 mmc 驱动未出现“mmc1: new high speed SDIO card at address 0001”，排查是 dts 中 mmc power 引脚有效电平配置错误，主控未能识别到 SDIO 设备，加入 power-inverted 属性解决。

6.2 问题 2

若已经正常运行的 hi3881 模组，设备断电，且 hi3881 模组一直处于供电状态，再次加载驱动会出现如下打印：

```

1 insmod ./hi3881.ko
2 [VERSION]:Hi3881V100R001C00SPC021 2020-11-11 15:10:00
3 oam_main_init SUCCESSFULLY!
4 wal_customize_init SUCCESSFULLY!
5 oal_main_init SUCCESSFULLY
6 frw_main_init SUCCESSFULLY!
7 hi_wifi_plat_init SUCCESSFULLY
8 {alloc 131072 bytes tx_flat_buff!}
9 {alloc 131072 bytes rx_flat_buff!}
10 hcc_hmac_init SUCCESSFULLY
11 plat_firmware_init SUCCESSFULLY
12 12r:[3]..
13 done
14 [ERROR]:839:check version FAIL [-1]
15
16 [ERROR]:76:firmware_download_function:: firmware download fail
17 [ERROR]:92:wlan_power_on:: firmware download fail
18 [ERROR]:405:wlan_pm_open:: wlan_power_on fail!
19 [ERROR]:278:wlan_pm_open error
20
21 [ERROR]:182:wifi host initialize fail.
22 frw_main_exit SUCCESSFULLY
23 hcc_host_exit start
24 hcc_delete_heartbeat_timer
25 [ERROR]:177:{oal_sdio_send_msg::failed to send sdio msg[4]!ret=-
26 110}
27 oal_sdio_dev_deinit!
28 oal_free_sdio_stru
29 Hisilicon connectivity sdio driver has been removed.sdio module
30 unregistered
31 hcc_host_exit finished
32 oal_main_exit SUCCESSFULLY
33 wal_customize_exit SUCCESSFULLY

```

Hi3881 没有完全复位，需要控制其复位引脚进行复位，或者将 Hi3881 模组完全断电。

6.3 问题 3

加载驱动过程中，日志提示 cmd 通信失败，可能是 dts 中 mmc 引脚配置错误，导致 sdio 通信失败，检查 dts 中的 mmc 配置。

6.4 案例分析

客户反馈加载 ak_mmc 驱动没有识别到 SDIO 设备，在开发板上验证当前版本 ak_mmc 及 hi3881 驱动是正常，加载 hi3881 后能够正常配网。

使用同一固件，同一 dts，手动控制 wifi 电源引脚(注：开发板与客户板 wifi 电源引脚 gpio 不一样，为减少干扰因素，使用同一 dts 进行验证)，无法正常识别到 SDIO 设备，按照《基于 HI3861L_HI3881 的 SDIO 识别问题案例.docx》文档所有描述的排查方式，焊接开发板及客户板上的 hi3881 串口。

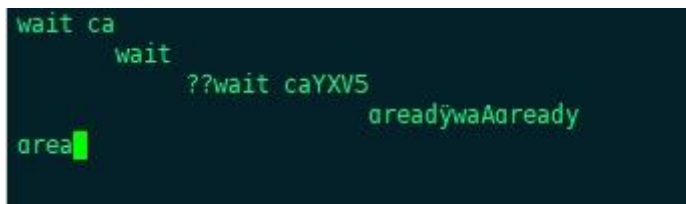
正常识别 SDIO 设备的开发板，Hi3881 芯片上电时会输出：

1 wait card ready

加载完 ak_mci 驱动，并识别到 sdio 设备，会输出：

1 wait ok

客户板 hi3881 上电串口输出乱码，加载 ak_mci 驱动，串口无任何信息。



```
wait ca
wait
??wait caYXV5
areadywaAready
area
```

定位是 hi3881 芯片硬件电路设计存在异常，经客户排查，由于 GPIO 共用了，导致 SDIO 通信异常，识别不到 SDIO 设备，调整硬件后正常识别。

7 附录:

参考文档:

1. 涂鸦模组需求导入确认书模板 - HI3881.docx
2. Hi3881V100 Linux 软件 使用指南.pdf
3. Hi3881V100 Linux 驱动 移植指南.pdf
4. AnyCloud39EV330 PDK 内核板级配置手册_V1.0.1.pdf
5. AnyCloud39EV330 PDK 开发板使用说明书_V1.0.1.pdf
6. AnyCloud39EV330 PDK 用户开发手册_V1.0.1.pdf
7. 基于 HI3861L_HI3881 的 SDIO 识别问题案例.docx